

Streaming Live RAG Implementation Summary

We have successfully executed the final implementation plan (v3) to address all outstanding gaps and bugs for the Streaming Live RAG prototype.

Completed Phases

1. Phase 1: Failure Modes & LLM Fallbacks

- Implemented robust `_asyncio.wait_for` retry wrappers with exponential backoff (1.5x) in `AnswerSynthesizer` (8s timeout) and `IntentDecomposer` (6s timeout).
- If the decomposer fails (e.g. rate limit), it degrades gracefully to single-intent mode instead of crashing, emitting `fallback: True`.
- Added unhandled exception fallbacks in the FastAPI WebSocket server so the socket remains open and errors are communicated clearly.
- `test_failure_fallback.py` chaos tests all pass.

2. Phase 2: Thread-Safe Concurrency

- Initialized `threading.Lock()`s for both `_EmbedModel` and `_CrossEncoder` in `HybridRetriever`.
- Applied the locks within `run_in_executor` blocks to ensure model inference is thread-safe across concurrent WebSockets.
- Created `test_concurrency.py` tests simulating 20 simultaneous connections, proving no state leakage or thread contention.

3. Phase 3: Confidence Calibration

- Built a 30-utterance labeled `calibration_set.json` spanning WAIT, RETRIEVE, and SUPPRESS intents.
- Integrated a new `run_calibration()` check into `scripts/measure_metrics.py`. It groups predictions by confidence bucket and verifies that accuracy aligns with confidence within a 15% tolerance gap.

4. Phase 4: Open-Vocabulary Constraints

- Updated `StateManager` to fall back to an LLM extraction call (`_llm_extract_constraint`) if regex rules miss a filter.
- Locked extraction schema exclusively to valid `_CORPUS_META_FIELDS` (year and domain), explicitly excluding internal fields like `source`.
- Made `detect_late_constraints` async.

5. Phase 5: Cross-Sub-Query Contradiction Handling

- Implemented `_check_contradictions` in `AnswerSynthesizer` to perform NLI (entailment vs contradiction) across the top retrieved chunk from *each distinct sub-query*.
- Updated the WebSocket server to emit a `conflicting_evidence` event to the client UI when a contradiction is found.
- Dynamically appends a prompt suffix instructing the synthesizer to present *both*

conflicting facts and explicitly note the discrepancy.

6. Model Upgrades & Project Readme

- Updated the model to Gemini 3.6 Flash ('gemini/gemini-3.6-flash'). All direct `openai.chat.completions` calls have been replaced with `litellm` to preserve the free-tier model requirements without limiting future adaptability.
- Updated `README.md` to explicitly state Known Non-Goals (e.g. ASR robustness, barge-in) and Failure Modes.

All `pytest` tests pass (41/41), and the `evaluate.py` gating pipeline returns **PASS** for all components. The system is fully ready for deployment.